Studio Nx

백엔드 274 — 클립 모델

2026-10-19

백엔드 274 — 클립 모델 (models/clip.py)

항목	내용
파일	`studionx/backend/app/models/clip.py`
작업	S1-10a 클립·연결·패턴 API: 타임라인 클립 표 모델 (PRD 02 FR-2196)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 273 `sequencer-devices.js`, 다음 275 `TrackConnection` 모델

1. 하는 일

트랙 위에 놓인 클립 한 건을 담는 SQLAlchemy 모델입니다. 모델은 실제 MySQL 표 `clips` 와 1:1 로 맞춰져 있고, `studionx/db/02_phase2_sequencer.sql` 과 `docs/DB.md` §3.3 의 정의와 열 이름이 같습니다.

```
clips 한 행 (트랙 위 클립)
├ clip_type = MIDI  — pattern_id 필수, sample_id 비움. 노트는 패턴(midi_notes)에 있음
├ clip_type = AUDIO — sample_id 필수, pattern_id 비움
├ 위치와 길이       — start_tick, length_tick, offset_tick (1박 = 960 tick)
└ 소리 모양 칸      — name, gain_db, is_looped, fade_in_tick, fade_out_tick,
                     fade_in_curve, fade_out_curve, warp_mode, warp_markers,
                     pitch_semitones, is_reversed
```

트랙을 지우면 DB 가 `ON DELETE CASCADE` 로 클립도 함께 지웁니다. 반대로 패턴과 샘플에는 `ON DELETE` 가 없어서 DB 가 RESTRICT 로 막습니다. 그래서 클립이 가리키는 패턴은 지울 수 없습니다.

쓰는 곳은 `clip_service` 의 목록, 만들기, 수정, 삭제, `api/v1/clips.py`, `full_project_service` 가 채우는 `/projects/{id}/full` 의 `clips` 목록, 그리고 패턴을 지우기 전에 확인하는 `pattern_is_used` 입니다.

2. 코드

```
class ClipType(StrEnum):
    AUDIO = "AUDIO"
    MIDI = "MIDI"


class WarpMode(StrEnum):
    """오디오 클립의 시간 늘리기 방식 (FR-2196). BEATS 는 박자를 맞춰 늘린다."""

    OFF = "OFF"
    BEATS = "BEATS"
    TONES = "TONES"
    TEXTURE = "TEXTURE"
    REPITCH = "REPITCH"
    COMPLEX = "COMPLEX"


class Clip(Base):
    """타임라인 클립 한 건. MIDI 는 pattern_id, AUDIO 는 sample_id 를 가진다."""
```

```
    __tablename__ = "clips"

    clip_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    track_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE")
    )
    clip_type: Mapped[ClipType] = mapped_column(Enum(ClipType))
    sample_id: Mapped[int | None] = mapped_column(BigInteger, ForeignKey("samples.sample_id"))
    pattern_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("patterns.pattern_id")
    )
    name: Mapped[str | None] = mapped_column(String(100))
    start_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True))
    length_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True))
    offset_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True), server_default=text("0"))
    gain_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    is_looped: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    fade_in_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True), server_default=text("0"))
    fade_out_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True), server_default=text("0"))
    fade_in_curve: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    fade_out_curve: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    warp_mode: Mapped[WarpMode] = mapped_column(
        Enum(WarpMode), server_default=WarpMode.BEATS
    )
    warp_markers: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    pitch_semitones: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    is_reversed: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
```

문법 풀이

식	뜻
`__tablename__ = "clips"`	이 모델이 실제 표 `clips` 를 담당함을 적습니다
`Mapped[int]` 와 `mapped_column(...)`	파이썬 칸에 DB 열의 자료형과 기본값을 함께 적는 최신 표기법입니다
`ForeignKey("tracks.track_id", ondelete="CASCADE")`	트랙을 지우면 DB 가 이 클립도 함께 지웁니다
`ForeignKey("samples.sample_id")` 와 `ForeignKey("patterns.pattern_id")`	`ON DELETE` 가 없어 DB 가 RESTRICT 로 막습니다. 클립이 쓰는 샘플과 패턴은 지울 수 없습니다
`Mapped[int | None]`	비어 있을 수 있는 칸입니다. 종류에 따라 `sample_id` 와 `pattern_id` 중 하나만 값이 들어갑니다
`Enum(ClipType)`	`"AUDIO"` 와 `"MIDI"` 두 글자만 받는 DB ENUM 입니다
`StrEnum`	열거형을 글자 그대로 비교하게 합니다. `clip.clip_type == "MIDI"` 가 참입니다
`server_default=text("0.00")`	값을 안 보내면 DB 가 기본값을 채웁니다

식	뜻
`Numeric(5, 2)`	소수 둘째 자리까지의 숫자입니다. `gain_db` 와 `pitch_semitones` 가 이 형태입니다
`INTEGER(unsigned=True)`	tick 은 음수가 되지 않으므로 0 부터 약 42억까지 씁니다
`server_onupdate=FetchedValue()`	행을 고치면 `updated_at` 을 DB 가 알아서 바꿉니다
`JSON`	`warp_markers` 처럼 모양이 정해지지 않은 값을 그대로 담습니다

3. 왜 이렇게

선택	이유
종류 규칙을 모델이 아니라 DB CHECK 에 둠	`chk_clips_kind` 가 MIDI 는 `pattern_id` 만, AUDIO 는 `sample_id` 만 갖게 합니다. DB 가 마지막으로 지키므로 어떤 코드 경로로도 어긋난 행이 들어가지 않습니다
같은 규칙을 요청 스키마가 먼저 검사	`ClipCreateRequest._kind_matches_ids` 가 422 로 막아 DB 까지 가지 않습니다 (276 문서)
`track_id` 에만 CASCADE	트랙이 곧 클립의 집입니다. 트랙이 지워지면 거기에 놓인 클립도 쓸모가 없습니다
`pattern_id` 와 `sample_id` 에 ON DELETE 를 안 씀	패턴과 샘플을 지우면 노트와 소리가 통째로 사라집니다. 클립이 없는 패턴과 쓰이지 않는 샘플만 지울 수 있게 RESTRICT 로 막았습니다
fade 칸과 warp 칸을 지금부터 넣음	Phase 2 재생 확장용입니다. 나중에 표를 바꾸지 않도록 칸과 기본값을 미리 맞춰 둡니다
`created_at` 과 `updated_at` 을 `DATETIME(fsp=3)` 로	DB 표 정의와 같고, 밀리초까지 기록합니다
클래스에 `__table_args__` 없음	CHECK 와 인덱스는 SQL 쪽(`02_phase2_sequencer.sql`)에 이미 있으므로 모델에 다시 쓰지 않습니다

4. 검증

`tests/test_clip_model.py` 다섯 건이 모두 통과합니다. `test_clip_columns_match_db` 는 칸 이름, 순서, NULL 허용, FK 를 실제 MySQL 표와 대조하고(assert_matches_db), `test_midi_clip_defaults` 와 `test_audio_clip_defaults` 는 종류별 기본값, `test_deleting_track_deletes_clips` 는 CASCADE, `test_pattern_used_by_clip_cannot_be_deleted` 는 RESTRICT 로 인한 `IntegrityError` 를 확인합니다. `tests/test_pattern_routes.py` 의 `test_delete_pattern_used_by_clip_is_400` 은 클립이 쓰는 패턴 삭제가 400 으로 막히는지, `tests/test_full_project_routes.py` 의 `test_full_includes_clips_and_connections` 은 `/full` 응답에 클립이 담기는지 봅니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 함수 문서(274 부터 276 까지)가 모두 끝난 뒤 한 번에 검토합니다
DB 변경 없음	`clips` 표는 이미 로컬 MySQL에 있어 마이그레이션을 따로 실행하지 않습니다